

```

if ($conn -> more_results()) {
    printf("-----\n"); echo "<br>";
}
//Move to next record set
} while ($conn -> next_result());
}
$conn->close();
?>

```

This code will get its input variable through `$_POST` and will make a query statement `$sql`. If the input variable contains multiple SQL statements separated by ‘;’ then the query statement `$sql` will be divided into separate query strings by the `multi_query()` method of the connection object `$conn`. The stored results of each SQL statement will be fetched and displayed on the screen by an iterative `while` loop.

```

if ($result = $conn -> store_result()) {
    while ($row = $result -> fetch_row()) {
        //Display only first two fields
        printf("%s,%s\n", $row[0], $row[1]); echo "<br>";
    }
}

```

At the end of a record set of a stored result, the loop will iterate to the next stored result set and the browsing on the selected tables will continue for all the successfully executed stored result sets. For example, if the form input with three SQL queries is:

```
parker'; select * from users; select * from indentreg
```

...then the above PHP routine executes all of them in three iterations of the `do-while` loop. First, the record sets of the user’s table for user name `parker` will be displayed. In the subsequent two iterations, it will browse through the users’ and `indentreg` tables. In each iteration, queries will display only the first two fields of the record sets.

```

115, parker
135, parker
-----
100,dipankar
110,avishek
120,demo
115,parker
...
...
-----
1,003
2,002
3,001
4,005

```

Figure 6: Multiple query input form

This problem of injecting multiple SQL statements to the currently active database on the server associated with an open file pointer is not there in `mysqli_query()`, as it does not support multiple statements.

Prevention: The best remedy is the restriction of the length of the input field. Next, verify the input string for data type and strictly try to adhere to the required format. Finally, the input fields should be specific to table fields, and verification of each field should be done before passing the variable to its server-side execution platform. The server-side platform should also have a proper filtering mechanism to prevent any unfortunate incident within the database.

MySQL function `mysql_real_escape_string()` is a good option to prevent SQL injection vulnerability. It is used to escape special characters in a string to be used in an SQL statement. This function creates a legal SQL string that can be used in an SQL statement. With the help of this function, the given input string is encoded to produce an escaped SQL string. It takes the current character set of the connection into account while preparing a valid string. Here is an example of the use of this function:

```

$username = $conn->real_escape_string($_POST['username']) ;
$sql = "SELECT * FROM passwdOLD WHERE username = '$username'";
$result = mysqli_query($conn,$sql);
// output data of each row
while($row = $result->fetch_assoc()) {
    echo "<br>id: " . $row["userid"]. " - Name: " .
    $row["username"]. "          " . $row["status"]. "<br>";
}

```

With the help of this function any input field variable of the form ‘`or 1=1 or parker`’— will fail to produce a legal SQL statement and only `parker` will produce a proper result.

We have discussed the three most common SQL injection vulnerabilities that are a real threat to the database server. There are a few more types of SQL injections too that can cause serious problems. 

By: Dr Dipankar Ray

The author is a member of IEEE, IET (UK). He has more than 20 years of experience in open source versions of UNIX operating systems and Sun Solaris, along with experience in programming and project development. He is presently working on data analysis and machine learning using neural networks and different statistical tools.